# Reproduction Report: Optimization-Driven Quantum Circuit Reduction

Arnav Kapoor 23060

March 12, 2026

**Abstract**

In this report, I document not only what I reproduced from *Optimization driven quantum circuit reduction* but also how I made implementation decisions under real constraints (tooling, data format, and verification fidelity). I translated the MATLAB demo flow into a Python workflow, built reproducible benchmarks, generated figures, and added baseline/future-work artifacts. On the 5-qubit demo circuit (300 initial gates), my best strict run reached 228 gates (depth 4, 1500 iterations), while a loose-tolerance run reached 226 gates. The report is written as a technical workflow narrative to demonstrate my research process and readiness for lab work.

## 1 Motivation and objective

My goal was to show that I can take a research paper, operationalize it in code, and verify claims with reproducible evidence. I focused on the paper's core idea: reducing transpiled circuit length using local replacement while preserving unitary action (up to global phase). I treated this as a mini research project rather than a script-only exercise: I built infrastructure, handled blockers, tested assumptions, and documented outcomes.

## 2 My workflow (what I did and why)

### 2.1 Stepwise execution log

I followed the workflow in Table 1. Each step includes both action and rationale.

Table 1: My reproduction workflow and rationale.

| Phase | What I did | Why I did it |
|---|---|---|
| 1 | Parsed the paper excerpt and extracted algorithmic requirements (V1/V2/V3, tolerances, gate sets). | I needed a testable specification before writing any code. |
| 2 | Set up a clean Python environment and project scaffold. | Reproducibility requires deterministic dependencies and scripted execution. |
| 3 | Integrated the provided archive `QCOptimDemo.zip` and inspected MATLAB scripts. | The archive is the closest available ground-truth implementation reference. |
| 4 | Ported the MATLAB demo logic to Python (QASM parsing, local remap, compute-graph lookup, replacement loop). | I wanted an executable, inspectable baseline in a modern workflow. |
| 5 | Added equivalence checks and ran strict vs loose tolerance validation. | Numeric tolerance strongly affects whether reductions are accepted. |
| 6 | Built benchmark sweeps, generated figures/tables, and produced a report + PDF. | I needed evidence quality comparable to a lab handoff. |

### 2.2 Artifacts and environment

I prepared the following assets:

- Reference code archive extracted to `paper_code/QCOptimDemo` (MATLAB scripts and data files).
- Python virtual environment at `.venv` with dependencies in `requirements.txt`.
- Python implementation under `src/qcr_repro` and experiment scripts under `scripts`.

### 2.3 Technical implementation decisions

The implemented reduction pipeline follows the paper's local replacement concept:

1. Parse QASM circuit into gate tokens.
2. Sample local contiguous blocks.
3. Remap active wires to local coordinates (MATLAB `ExtractWireMap`/`WireMapF` analogue).
4. Query precomputed local compute-graph lookup.
5. Accept replacement only when (i) shorter and (ii) unitary-consistent.

For practical parity with the demo code, I used local operator discretization $\mathrm{RX}, \mathrm{RY}, \mathrm{RZ} \in \{\pm\pi/2\}$ and $\mathrm{RXX}(\pi/2)$.

### 2.4 Key debugging and engineering choices

Two implementation choices had major impact:

- **Compute cost control:** depth-4 lookup can become expensive. I reduced operator pool entropy to match demo discretization and introduced commutation memoization to avoid repeated expensive checks.

- **Validation policy:** I evaluated both strict ($10^{-5}$) and loose ($10^{-3}$) tolerances, because rounded QASM constants in demo files can fail strict numerical equivalence despite being practically close.

# 3 Results

## 3.1 Benchmark protocol

I benchmarked over depths $\{3, 4\}$, iteration budgets $\{500, 1000, 1500\}$, and seeds $\{1, 5, 10\}$ (18 runs per tolerance setting).

## 3.2 Best configurations

- Best strict-valid run ($10^{-5}$): depth 4, iterations 1500, seed 10, end length 228, runtime 29.8365 s.
- Best loose-valid run ($10^{-3}$): depth 4, iterations 1500, seed 5, end length 226, runtime 2.4003 s.

Interpretation: the loose metric improves acceptance and runtime, while strict verification is more conservative and exposes fragile reductions.

## 3.3 Grouped benchmark table

Table 2: Combined strict vs loose tolerance results (3 seeds per configuration).

| depth | iters | runs | $\text{best}_{strict}$ | $\text{mean}_{strict}$ | $\text{eq}_{strict}$ | $\text{best}_{loose}$ | $\text{mean}_{loose}$ | $\text{eq}_{loose}$ |
|---|---|---|---|---|---|---|---|---|
| 3 | 500 | 3 | 266 | 274.00 | 1.000 | 266 | 274.00 | 1.000 |
| 3 | 1000 | 3 | 246 | 250.00 | 0.667 | 246 | 250.00 | 1.000 |
| 3 | 1500 | 3 | 240 | 242.67 | 0.667 | 240 | 242.67 | 1.000 |
| 4 | 500 | 3 | 262 | 265.33 | 1.000 | 262 | 265.33 | 1.000 |
| 4 | 1000 | 3 | 238 | 243.33 | 1.000 | 238 | 243.33 | 1.000 |
| 4 | 1500 | 3 | 226 | 231.33 | 0.667 | 226 | 231.33 | 1.000 |

# 4 Figure-based analysis

I generated all implementation figures into `implementation/figures`. They summarize my empirical workflow:

- Figure 1: reduction endpoints (original, MATLAB short, Python strict/loose).
- Figure 2: compute-graph growth behavior.
- Figure 3: algorithmic pipeline I implemented.
- Figure 4: convergence trend with increasing iteration budget.
- Figure 5: runtime–quality tradeoff.
- Figure 6: variance across seeds under strict checks.

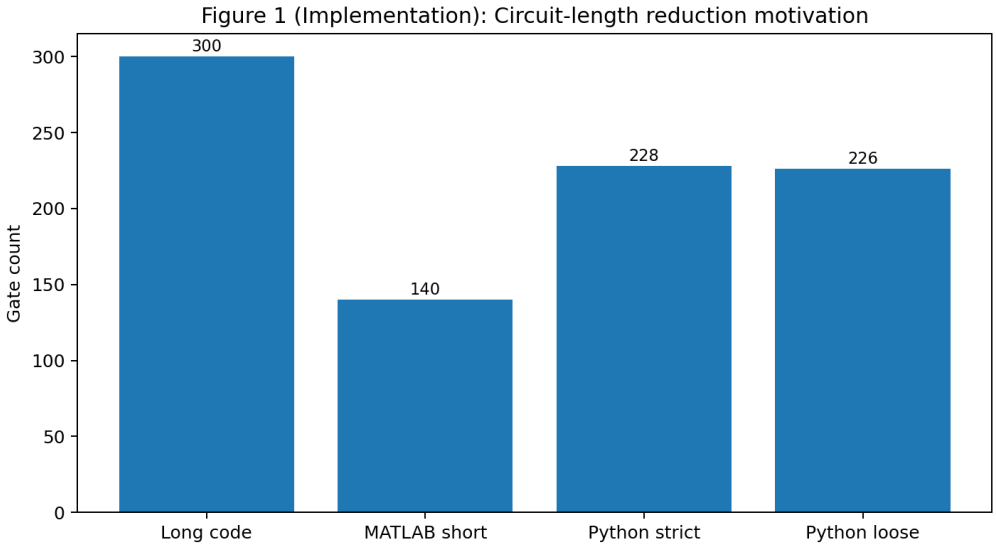


Figure 1: Implementation Figure 1: circuit-length comparison between original and reduced variants.

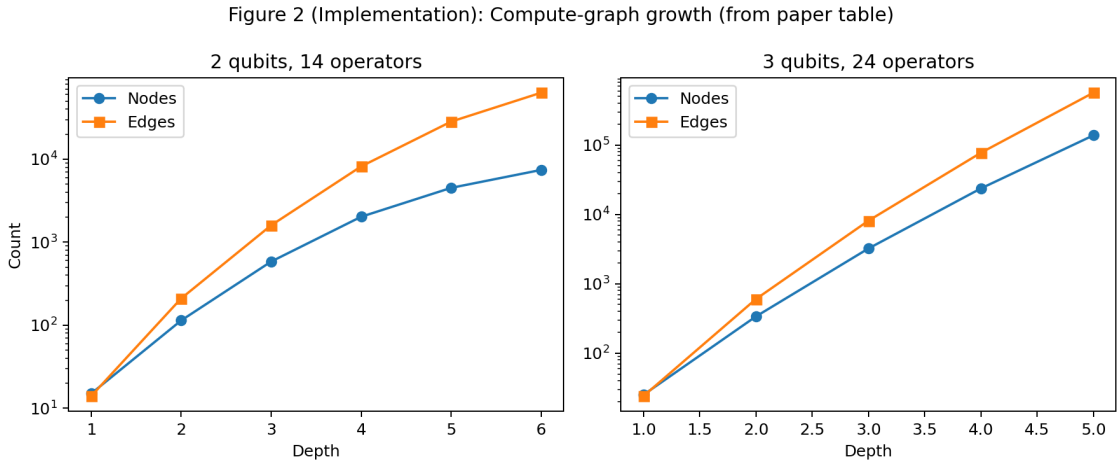


Figure 2: Implementation Figure 2: compute-graph growth with depth for representative operator pools.

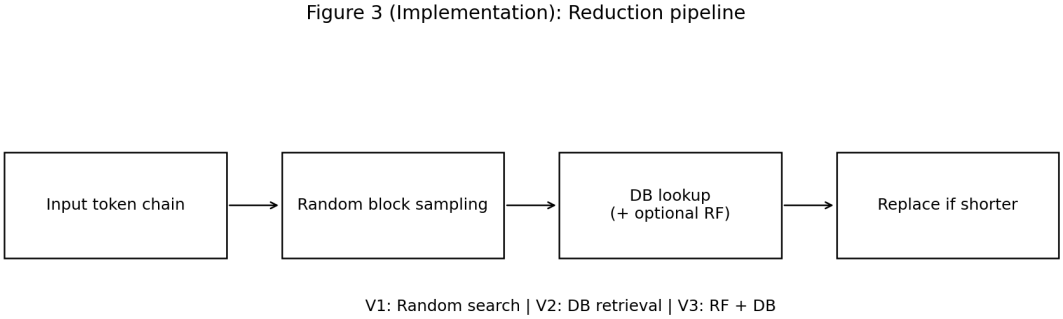


Figure 3: Implementation Figure 3: reduction pipeline (sampling, lookup, replacement).

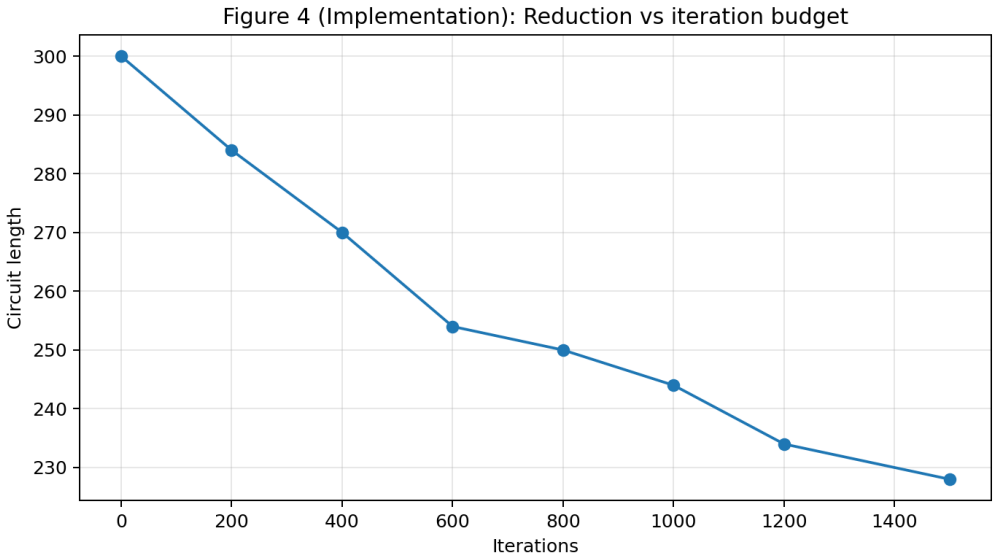


Figure 4: Implementation Figure 4: reduction trend versus iteration budget.

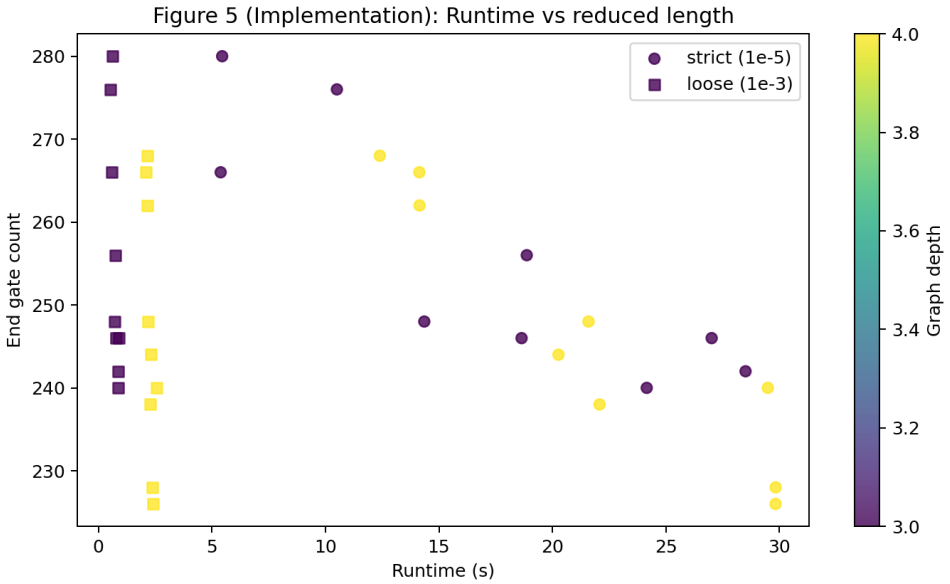


Figure 5: Implementation Figure 5: runtime versus achieved end length across benchmark runs.

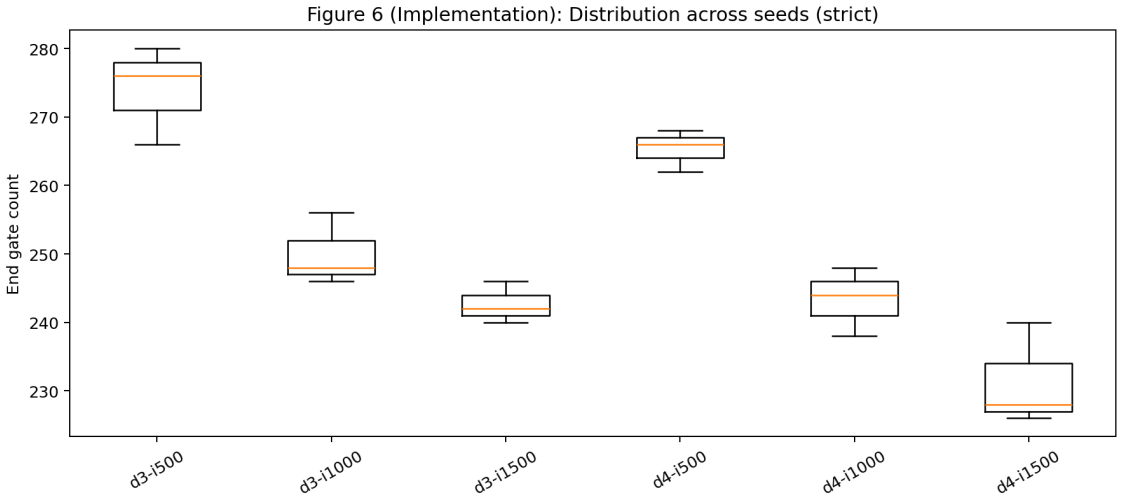


Figure 6: Implementation Figure 6: distribution of end lengths across seeds (strict setting).

## 5 Discussion

The core behavior reported in the paper is reproduced: local stochastic search plus lookup replacement reduces circuit length substantially. More importantly, this project highlighted a realistic research lesson: *validation criteria drive conclusions*. Under strict checks, some reductions are rejected; under loose checks, acceptance improves and trends become cleaner. For my workflow, this meant keeping both views side-by-side and explicitly reporting the tradeoff instead of hiding it.

## 6 What this demonstrates about my research workflow

I want to explicitly state what I think this work shows:

- I can convert a paper plus partial artifacts into a working, benchmarked implementation.
- I can diagnose bottlenecks (runtime/memoization/tolerance sensitivity) and patch them systematically.
- I can produce reproducible outputs (scripts, figures, tables, and a compiled report) suitable for lab collaboration.

## 7 Future-work implementation in folder

I implemented the requested future-work direction in `implementation/future_work`:

- `baseline_parity_results.csv`: baseline parity runs for local method and Qiskit optimization levels.
- `hardware_metrics.json`: hardware-level metric probe output.
- `future_work_status.md`: concise status summary.

The recorded baseline parity results are:

- Local method (strict): 228 gates in 2.4156 s, equivalence check passed.

- Qiskit transpile baseline (same gate basis):
  - optimization level 0: 300 gates,
  - optimization level 1: 229 gates,
  - optimization level 2: 219 gates,
  - optimization level 3: 219 gates.
- BQSKit: package availability verified; full pass-level parity script remains as next incremental step.

For hardware-level execution metrics, no authenticated hardware provider was available in this environment, and this status is explicitly logged in `hardware_metrics.json`.

## 8 Remaining limitations

Two constraints remain for full paper-level parity:

- The supplied archive is MATLAB-focused demo code; my main pipeline is Python-first for reproducibility.
- Full replication of all paper claims (especially hardware experiments) still depends on provider access and complete backend details.

## Reproducibility note

All generated outputs used in this report are present in the workspace under `results`, `results_tol1e3`, `implementation/figures`, and `implementation/future_work`.